

课程笔记

CME 295 第 7 讲: Agentic LLMs

五道口纳什 & Codex

2025-11-14



视频作者/频道: Stanford Online

发布日期: 2025-11-18

视频时长: 1:49:22

视频链接: <https://www.youtube.com/watch?v=h-7S6HNq0Vg&list=PLoROMvodv4r0CXd21gf0CF4xr35yINe0y&index=7>

目录

1 从推理模型到 Agentic LLM：问题为什么发生了变化	2
1.1 本章小结	2
2 RAG：把静态语言模型接到动态知识库	2
2.1 为什么需要 RAG	2
2.2 RAG 的三步工作流	3
2.3 知识库构建与候选召回	3
2.4 重排序、精度提升与检索评测	4
2.5 本章小结	5
3 Tool Calling：让模型不仅会说，还能动手	5
3.1 从非结构化 RAG 到结构化 API	5
3.2 如何把工具暴露给模型	5
3.3 工具的典型用途与扩展问题	6
3.4 标准化协议：MCP 的意义	6
3.5 本章小结	7
4 Agents：把多次思考、调用与观察串成闭环	7
4.1 什么叫 agent，而不仅仅是多工具调用	7
4.2 ReAct：Reason + Act	8
4.3 多 agent 协作、A2A 与安全	8
4.4 对实践者的最后建议	8
4.5 本章小结	8
5 总结与延伸	8

1 从推理模型到 Agentic LLM：问题为什么发生了变化

本讲开头先回顾上一讲的 reasoning models。课程前半段已经说明：普通 decoder-only LLM 的强项是模仿、生成和代码补全，但其弱点同样明确，包括推理能力有限、知识停留在预训练截止时间、无法直接执行动作，以及系统级评估很困难。上一讲通过 GRPO、DAPO 等强化学习方法缓解了“推理不足”这个短板；而这一讲要处理的是另外两类更工程化的问题：模型知识如何接入外部世界，模型又如何代表用户执行动作。

讲者因此把整讲压缩成三条主线：RAG、tool calling、agents。它们并不是彼此独立的三个 buzzword，而是同一条系统能力升级路径上的三个层次。RAG 解决“模型不知道最新信息”的问题；tool calling 解决“模型虽然知道该做什么，但自己动不了手”的问题；agents 则把多次检索、思考、调用工具、观察结果串成闭环，让系统从“一次回答器”变成“多步任务执行器”。

本讲总问题

如果把 LLM 只当成一个从 prompt 到 response 的函数，那么它再强也只能在权重和上下文窗口的边界内工作。Agentic LLM 的核心，就是把模型嵌进外部知识、外部工具和外部状态构成的系统里。

1.1 本章小结

这一节建立了整讲的总视角：上一讲解决的是模型内部推理，本讲解决的是模型与外部世界的连接。后面三部分可以分别理解为“接知识”“接能力”“接工作流”。

2 RAG：把静态语言模型接到动态知识库

2.1 为什么需要 RAG

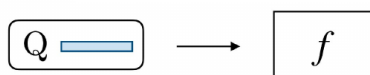
讲者先从最直接的失败案例出发：模型的预训练数据如果停在一个月前，那么它自然不知道两周前发生的选举结果、最新发布的产品规格，或者某个仓库刚提交的补丁。继续训练当然是一种办法，但在实践里有三个明显缺点。

第一，持续改写模型知识可能带来别的能力退化。第二，很多应用已经在基础模型上做了额外微调，如果每次知识更新都要把这些派生模型重新处理一遍，维护成本会急剧上升。第三，即便不考虑训练成本，把大量新材料直接塞进上下文也不现实，因为上下文长度有限，而且长上下文会带来注意力分散与 token 计费问题。

Overview

RAG = Retrieval-Augmented Generation

Idea. Augment prompt with **relevant** pieces of information.



Stanford University

Figure from "Super Study Guide: Transformers & Large Language Models", Amidi et al., 2024.

ICME

图 1: RAG 的核心思想: 先检索与问题真正相关的材料, 再把它们拼进 prompt, 让语言模型在“带证据”的上下文里生成答案。

因此, RAG 的本质不是“让模型记住更多东西”, 而是“让模型在推理时临时读到需要的东西”。这一点很重要, 因为它把知识更新问题从 expensive model update 变成了 cheaper knowledge-base maintenance。

2.2 RAG 的三步工作流

课程把 RAG 概括成 retrieve, augment, generate 三步:

第一步, 检索。给定用户问题, 在知识库找出可能相关的文档块。第二步, 增强。把检索结果和原始问题放在一起, 拼成新的输入。第三步, 生成。LLM 只基于这份增强后的上下文生成答案。

这个过程里最容易被低估的是“第一步”。如果检索候选已经错了, 后面的 prompt engineering 和更强模型都只是“在错误证据上认真思考”。所以讲者明确强调: RAG 的瓶颈通常不在生成, 而在 retrieval stage。

2.3 知识库构建与候选召回

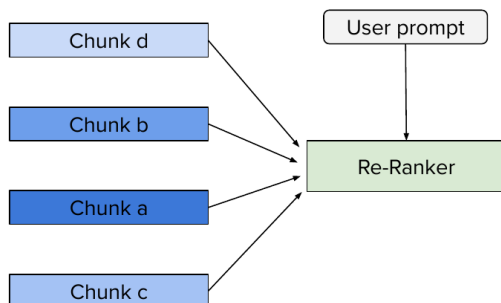
在真正检索之前, 系统要先把原始资料加工成可检索对象: 收集文档、做切分、为 chunk 计算 embedding、必要时建立倒排索引或混合索引。讲义里可以把这一过程视为“把非结构化资料重排成可搜索知识库”。

候选召回阶段的目标是 **最大化 recall**, 即宁可多拿一些可能相关的候选, 也不要真正把有用的片段漏掉。课程给出三类典型方法:

1. 语义检索: 把 query 和 chunk 分别编码为向量, 用向量相似度找候选。典型结构是 bi-encoder。
2. 关键词检索: 如 BM25, 特别适合实体名、缩写、罕见术语、日志关键字等 lexical match 很强的场景。
3. 混合检索: 语义相似度与关键词匹配联合使用, 兼顾 recall 和稳健性。

Step 2: ranking

Objective. Provide final relevance score using more sophisticated (re-)ranker



Stanford University

ICME

图 2: RAG 的第二阶段重排序: 先用低成本检索拿到候选集, 再让更重的 re-ranker 对少量候选重新打分。

讲者还补充了几类能显著提升首轮召回质量的工程技巧。其一, 是让 query embedding 更贴近真实检索意图, 例如通过 instruction-style query reformulation 缓解 query 和 document embedding 分布不一致的问题。其二, 是对文档切片做 contextualization, 让每个 chunk 不只是孤立片段, 而是带有来自原文的必要定位信息。其三, 是 prompt caching 一类系统优化, 用来降低重复检索和重复上下文注入的成本。

2.4 重排序、精度提升与检索评测

候选召回之后, 系统进入 ranking 或 reranking 阶段。这里的目标不再是 recall, 而是 **precision**: 从已经拿到的小规模候选里, 尽量把最相关的几条放在最前面。课程用 cross-encoder 作为典型例子, 强调这类模型会把 query 与 candidate jointly encode, 再输出更精细的相关性分数, 因此质量更高, 但成本也明显更高。

这也解释了为什么工业系统常用“两阶段检索”:

1. 先用便宜的召回器在大库里快速缩小范围。
2. 再用更贵的 reranker 在小集合里做精排。

课程还讨论了如何量化 retrieval 质量。对于“前 k 个结果里是否包含相关 chunk”的问题, 可以用 $\text{recall}@k$ 或 $\text{precision}@k$; 如果还关心排序位置, 则常见指标是 $\text{NDCG}@k$ 。其思想是: 越靠前出现的相关结果, 贡献越大。可写成

$$\text{DCG}@k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}, \quad \text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}.$$

其中:

- rel_i 表示第 i 个结果的相关性标签;

- $IDCG@k$ 表示理想排序下的最大 DCG；
- NDCG 越接近 1，说明系统越能把高相关结果排在前面。

RAG 的工程准则

RAG 系统不是“把更多文本喂给模型”，而是“在有限上下文预算里，让最相关、最可信、最可用的证据排到前面”。因此，召回与精排必须分开设计、分开评估。

2.5 本章小结

RAG 解决的是语言模型知识静态、上下文昂贵、长上下文易分心的问题。其实践重点不在“把模型变聪明”，而在“把检索系统做好”：先构建知识库，再高 recall 地召回候选，最后用重排序和指标体系把有效证据放到最前面。

3 Tool Calling：让模型不仅会说，还能动手

3.1 从非结构化 RAG 到结构化 API

如果说 RAG 把“文本化知识”接入模型，那么 tool calling 接入的是“可执行能力”。课程用一个很朴素的例子说明差别：用户说“帮我找附近的 teddy bear”。纯 LLM 只能给出泛泛建议，而带工具的 LLM 可以调用地理定位、商店搜索或数据库查询接口，返回真正可执行的结果。

这一点意味着工具不是普通上下文，而是带有参数结构、输入输出约束、乃至真实副作用的能力单元。模型的任务不再只是续写文本，而是：

1. 识别当前问题是否需要工具；
2. 选择正确工具；
3. 生成正确参数；
4. 读取工具返回值；
5. 基于返回值继续回答或进入下一步。

3.2 如何把工具暴露给模型

课程给出两种主流方式。第一种是训练式方法：在监督微调或后续训练中，让模型反复见到“何时调用哪个工具、参数如何填写、工具输出如何并回对话历史”的样本。第二种是提示式方法：在上下文中直接提供函数签名、字段说明和调用说明，让模型通过 in-context learning 学会使用。

下面这个最小伪代码，正对应课程里的 `find_teddy_bear` 示例：

```
1 from typing import TypedDict
2
3 class TeddyBearInfo(TypedDict):
4     name: str
5     address: str
6     distance_km: float
7
8 def find_teddy_bear(location: tuple[float, float]) -> TeddyBearInfo:
```

```
"""Return the nearest teddy-bear store for a given location."""
```

Listing 1: 课程中的工具 API 思路，可视为对 slide 示例的压缩重述

这个接口本身并不神秘，难点在于让 LLM 可靠地把自然语言目标映射成结构化参数，再把后端输出重新翻译成用户能理解的回答。也正因如此，tool calling 往往是 prompt design、schema 设计和后端可靠性共同作用的结果，而不是模型单方能力。

3.3 工具的典型用途与扩展问题

课程把工具用途分成若干典型类别：信息获取，例如网页、数据库、代码库搜索；计算型任务，例如单位换算、财务计算、调用外部求解器；以及执行型任务，例如发邮件、下单、改数据库、调节设备。这三类难度逐步上升，因为从“读”到“算”再到“写”，错误代价越来越大。

当工具数量上升时，又会出现新问题：把所有工具定义同时塞进上下文会增加延迟，也会稀释模型对每个工具说明的注意力。讲者因此引出了 **tool selection** 或 routing：先用一层较轻的选择器筛出少数相关工具，再把这些工具的详细 schema 提供给主模型。

3.4 标准化协议：MCP 的意义

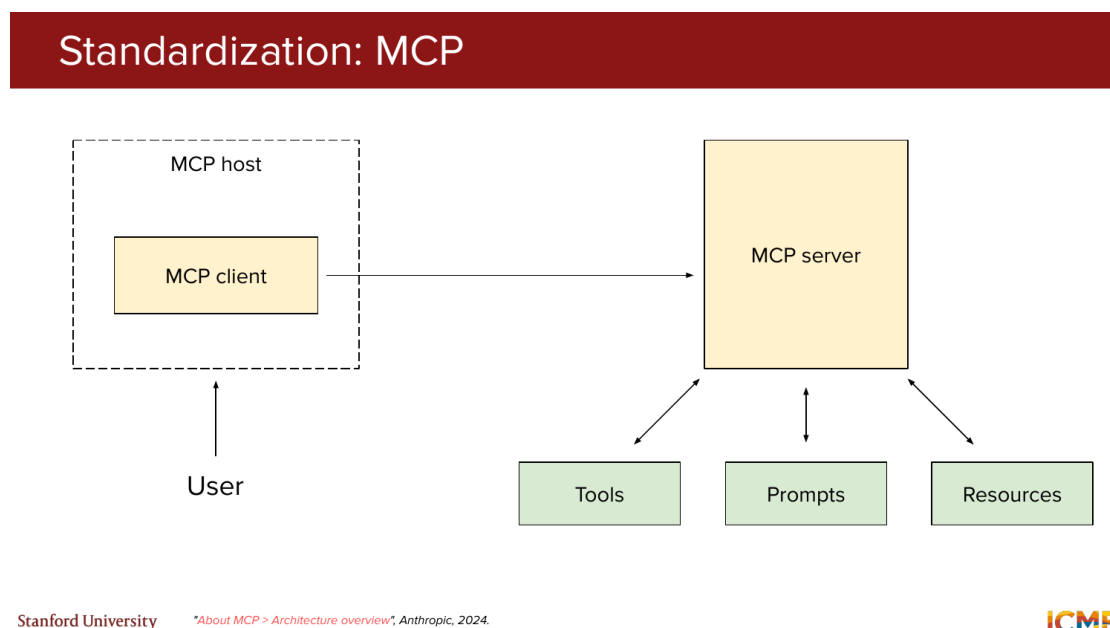


图 3: MCP 的课程示意图：host、client、server 分层，把工具、资源与 prompt 暴露方式标准化。

当不同模型、不同宿主、不同工具实现都各自定义 schema 时，集成成本会非常高。课程因此把 MCP 介绍为一个关键标准化方向。它的思想不是重新发明工具，而是统一“模型如何发现工具、如何读取资源、如何调用能力”的接口约定。

从系统角度看，MCP 的价值主要体现在三点：

1. 复用性：同一批工具不必为每个 LLM 宿主重复接入。
2. 可组合性：工具、prompt、resources 能通过统一协议被声明和发现。
3. 可维护性：开发者把注意力从“接线差异”转移到“能力设计与权限治理”。

工具接入的真正难点

课程后半段一再暗示：tool calling 真正困难的地方不是“函数调用语法”，而是正确路由、正确参数、正确读取返回值，以及确保带副作用的操作可审计、可回滚、可限制权限。

3.5 本章小结

Tool calling 把 LLM 从“生成器”推进为“系统中的调度器”。它要求模型学会选择工具、构造参数、读取结果，并在工具越来越多时借助 router 与标准协议控制复杂度。MCP 的出现，正是为了降低这种复杂度的工程摩擦。

4 Agents：把多次思考、调用与观察串成闭环

4.1 什么叫 agent，而不仅仅是多工具调用

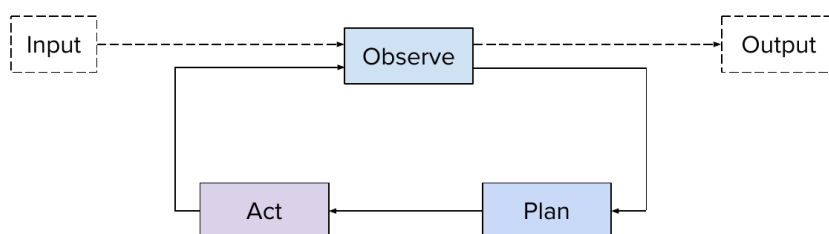
课程采用了一个足够实用的定义：agent 是一个会自主追求目标、代表用户完成任务的系统。与单次 tool call 相比，agent 的关键不在于“有没有工具”，而在于“有没有多步循环”。系统要能根据当前观察结果调整计划，再决定下一次行动。

因此，agent 可以理解为把以下四件事组装成循环：

1. 观察当前上下文与环境状态；
2. 形成阶段性计划；
3. 执行动作，如检索、调用工具、写入系统；
4. 读取结果，再进入下一轮。

Overview

ReAct = Reason + Act



Stanford University

"ReAct: Synergizing Reasoning and Acting in Language Models", Yao et al., 2022.

ICME

图 4: ReAct 框架：把 observe、plan、act 组织为闭环，是本讲给出的 agentic workflow 代表范式。

4.2 ReAct: Reason + Act

ReAct 是本讲最重要的 agent 框架。它的核心不是让模型“一口气想完所有步骤”，而是把 reasoning 与 acting 交替展开。模型先基于当前输入提出一个计划，再执行动作拿到新证据，再更新计划。这样做的好处是：模型无需在一开始就把整条路径想对，而是可以在执行过程中修正。

课程用“teddy bear 太冷了，请做点什么”这个例子说明：模型先意识到自己需要环境温度，调用测温工具；看到室温偏低后，再调用 thermostat 工具把温度调高；观察到成功返回后，才输出最终答复。这里 agent 的价值不是“比单工具更会调用”，而是能把多个工具调用和状态变化组织成一条可迭代工作流。

4.3 多 agent 协作、A2A 与安全

讲者随后把视角从单 agent 扩展到多 agent。不同 agent 可以分别负责占用检测、能耗管理、日程安排等子任务，问题随之变成：它们如何暴露能力，如何彼此发现，如何报告状态，如何处理中断与授权？课程把 A2A 作为对应的标准化方向，用来刻画“agent 与 agent 之间的契约”。

但一旦 agent 真正能执行外部动作，安全问题就立即升级。课程举的数据外泄与现实世界攻击案例都在说明：当系统既能读网页、又能操作工具、还能多轮规划时，失败不再只是“答案写错了”，而可能是“真的做了不该做的事”。

讲者给出的缓解思路分成两层：

1. 训练时防线：在 SFT 与后续训练中纳入 harmlessness / safety 数据；
2. 推理时防线：在执行前后加入 safety classifier、权限检查与额外守卫模型。

同时，他也给出很务实的工程建议：**start small, start smart, start correct**。先用最简单、最可观察的任务验证链路是否正确，再用最强模型测天花板，最后才去优化延迟与成本。

4.4 对实践者的最后建议

课程结尾把 agentic coding 视为最成熟、最值得学生立刻尝试的使用场景之一。原因很直接：它能替你完成管道化、重复性、跨文件的重负荷工作，释放认知带宽。但讲者同时强调，代码生成越来越便宜，真正稀缺的是判断代码是否正确、结构是否合理、系统是否安全的能力。

这句判断其实把整讲都串起来了。RAG、tool calling、agents 让模型触达更多信息与能力，但最终仍需要人类定义边界、验证结果、承担系统责任。

4.5 本章小结

Agent 不只是“会调很多工具的 LLM”，而是能够在观察、计划、行动之间循环的执行系统。ReAct 给出了最清晰的工作流模板；多 agent 与 A2A 则让这种能力走向协作化。但能力放大后，可靠性与安全也成为第一优先级。

5 总结与延伸

第 7 讲真正完成的是一次视角切换：语言模型不再被看成孤立的文本生成器，而被看成一个嵌入系统、接入外部知识、调用真实能力、在多步任务里迭代修正的计算节点。

从工程实现上看，本讲三条主线形成递进关系：

1. RAG 让模型读到最新、最相关的外部知识。
2. Tool calling 让模型把自然语言目标翻译成结构化 API 调用。
3. Agents 则把多次检索、推理、动作和反馈串成闭环。

从风险角度看,能力每前进一步,验证成本也同步上升。RAG 需要验证证据相关性; tool calling 需要验证路由、参数和返回值; agent 还要验证多步行为是否稳定、是否越权、是否安全。

如果把这讲内容转化为实践建议,可以归结为四句:

1. 检索系统先于 prompt engineering。
2. 工具 schema 与返回值设计和模型本身同样重要。
3. agent 工作流先求正确,再求便宜。
4. 生成越来越廉价,判断与约束越来越值钱。

沿着这条线往下走,下一讲自然会进入一个关键问题:既然系统越来越复杂,我们又该怎样评估它到底好不好、稳不稳、值不值得上线。